

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Sydney pursuant to Part VB of the Copyright Act 1968 (**the Act**). The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

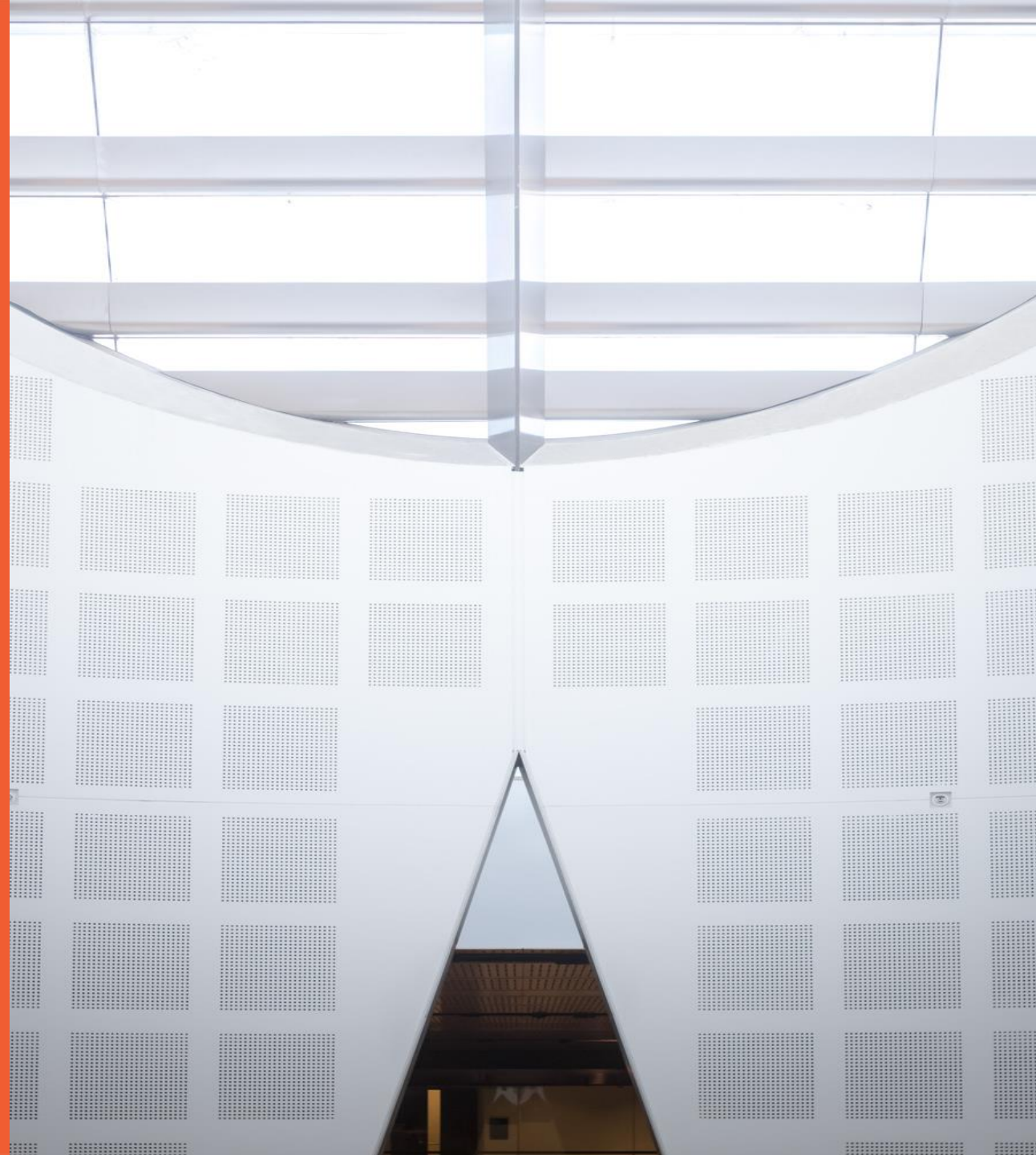
COMPx270: Randomised and
Advanced Algorithms
Lecture 10: Linear Programming
and Randomised Rounding

Clément Canonne

School of Computer Science



THE UNIVERSITY OF
SYDNEY



This week: Linear Programming, and Randomised Rounding

Maximize a linear function subject to linear inequality constraints on variables $x_1, \dots, x_n$ of interest.

Linear Programming

Maximize a linear function subject to linear inequality constraints on variables $x_1, \dots, x_n$ of interest.

or

maximise $\sum_{i=1}^n c_i x_i$ linear function
subject to variables

$\sum_{i=1}^n A_{ji} x_i \leq b_j, \quad 1 \leq j \leq m$
 $x_i \geq 0, \quad 1 \leq i \leq n$ linear constraints

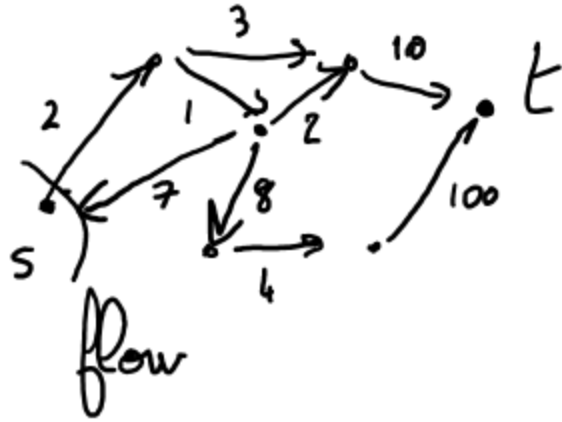
Equivalently:

$$\begin{aligned} &\text{maximize } c^T x \\ &\text{s.t. } Ax \leq b \end{aligned}$$

$(c_i, b_i, A_{ij} \text{ are parameters})$

Linear Programming

Example: Max Flow!



$G = (V, E)$ directed
 $s, t \in V$
 capacities $c: E \rightarrow \mathbb{R}_+$

Goal maximize $\sum_{u: (s,u) \in E} f_{su}$
 s.t. $0 \leq f_e \leq c(e) \quad \forall e \in E$
 $\sum_{v: (u,v) \in E} f_{uv} = \sum_{v: (v,u) \in E} f_{vu} \quad \forall u \in V \setminus \{s, t\}$

$$\text{maximise } \sum_{i=1}^n c_i x_i$$

subject to

$$\sum_{i=1}^n A_{ji} x_i \leq b_j, \quad 1 \leq j \leq m$$

$$x_i \geq 0, \quad 1 \leq i \leq n$$

Linear Programming

$$\text{maximise } \sum_{i=1}^n c_i x_i$$

subject to

$$\sum_{i=1}^n A_{ji} x_i \leq b_j, \quad 1 \leq j \leq m$$

$$x_i \geq 0, \quad 1 \leq i \leq n$$

- Everything in **P** can be phrased as an LP
- LPs can be solved in **polynomial time** (in the size of the input)
- LPs can be solved **efficiently** (in practice)
- We're not going to see **any of that**

↑ number of variables,
constraints,
size of coefficients

↓
if interested, take
COMP5530 / 4530 (Discrete Optimisation)
for more!

Linear Programming

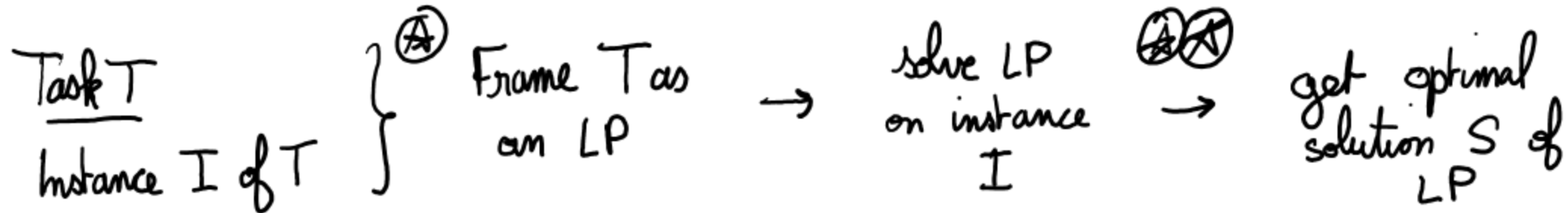
$$\text{maximise } \sum_{i=1}^n c_i x_i$$

subject to

$$\sum_{i=1}^n A_{ji} x_i \leq b_j, \quad 1 \leq j \leq m$$

$$x_i \geq 0, \quad 1 \leq i \leq n$$

Use them to solve problems either exactly or **approximately**.



$$\alpha \cdot \text{OPT}(I) \leq \text{VALUE}(S) \leq \text{OPT}(I)$$

$\alpha \in (0, 1]$ for maximization tasks
 the value of our solution S for I
 value of optimal solution for I
 (sometimes $\mathbb{E}[\text{value}(S)]$ if we do ⊛ or ⊛⊛ randomly)

Integer Linear Programming

$$\begin{array}{ll} \text{maximize} & \sum_{i=1}^n c_i x_i \\ \text{s.t.} & Ax \leq b \\ & x_i \in \{0, 1, \dots, k\} \quad \forall i \\ & (\text{typically } x_i \in \{0, 1\}) \end{array}$$

LP: $x_i \geq 0$ are possible constraints
 $x_i = 1$
not " $x_i = 1$ or $x_i = 0$ "
ILP: " $x_i = 1$ or $x_i = 0$ " is possible
(equivalent to $x_i^2 = x_i$)



More power!

But don't know how to solve them in general (efficiently)

Integer Linear Programming: st-Min-CUT

Digraph $G=(V, E)$ w/ capacities $c: E \rightarrow \mathbb{R}_+$
 $s, t \in V$



Minimize value
of the s-t cut
 $\text{value}(S) = \sum_{\substack{e=(u,v) \\ u \in S, v \in \bar{S}}} c(e)$

maximize $-\sum_{e \in E} c(e) x_e$
s.t.

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_{uv} \quad \forall (u, v) \in E$$

$$x_e, y_v \in \{0, 1\} \quad \forall e \in E$$

$$\forall v \in V$$

one variable
per edge

one variable
per vertex

Integer Linear Programming: st-Min-CUT

↖ we have poly-time
algs for this (e.g., Ford-Fulkerson)

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V$$

Nice, but don't know how to solve this
efficiently

LP Relaxation: st-Min-CUT

$$\begin{aligned} & \text{maximise} \quad - \sum_{e \in E} c_e x_e \\ & \text{subject to} \\ & \quad y_s = 0 \\ & \quad y_t = 1 \\ & \quad y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & \quad x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V \end{aligned}$$

$$\begin{aligned} & \text{maximise} \quad - \sum_{e \in E} c_e x_e \\ & \text{subject to} \\ & \quad y_s = 0 \\ & \quad y_t = 1 \\ & \quad y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & \quad x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V \end{aligned}$$

Nice, now we know how to solve this efficiently!
But it does not solve s-t-MinCut

LP Relaxation: st-Min-CUT

Fact 45.1. Let OPT_{ILP} be the optimal value of a solution to an ILP (maximisation problem), and OPT_{LP} be the optimal value of a solution to its LP relaxation. Then

$$\text{OPT}_{\text{ILP}} \leq \text{OPT}_{\text{LP}}.$$

(For a minimisation problem, the inequality is reversed.)



$$\begin{aligned} &\text{maximise } - \sum_{e \in E} c_e x_e && \text{OPT}_{\text{ILP}} \\ &\text{subject to} \\ &y_s = 0 \\ &y_t = 1 \\ &y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ &x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V \end{aligned}$$

$$\begin{aligned} &\text{maximise } - \sum_{e \in E} c_e x_e && \text{OPT}_{\text{LP}} \\ &\text{subject to} \\ &y_s = 0 \\ &y_t = 1 \\ &y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ &x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V \end{aligned}$$

ⓐ ⓑ
get feasible solution to T, S' → "did not lose much" when rounding
→ relate value of S' to value of S using constraints of LP

LP Relaxation: st-Min-CUT

① Solve LP relaxation optimally:
get (x^*, y^*) s.t. $\text{value}(x^*, y^*) = \text{OPT}_{\text{LP}}$

② "Round" y^* to get a good feasible solution
y to ILP.

⚠ "Naïve" rounding does not work well!

"If $y_u^* \geq 0.5$ set $y_u = 1$ "

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V$$

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V$$

we should use this!

LP Rounding!

~~Deterministic
rounding~~

Threshold
 $\bar{z} = 0.5$

Randomized
rounding

Random threshold
 $\tau \in [0, 1]$
(uniformly at random)

(Set $y_u = 1$
iff $y_u^* \geq \tau$)

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V$$

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V$$

$$y_v - y_u \leq x_e \quad \forall e = (u, v)$$

LP Rounding: st-Min-CUT

$$\begin{aligned}
 &\text{maximise} && - \sum_{e \in E} c_e x_e \\
 &\text{subject to} && \\
 &&& y_s = 0 \\
 &&& y_t = 1 \\
 &&& y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\
 &&& x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V
 \end{aligned}$$

$$\begin{aligned}
 &\text{maximise} && - \sum_{e \in E} c_e x_e \\
 &\text{subject to} && \\
 &&& y_s = 0 \\
 &&& y_t = 1 \\
 &&& y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\
 &&& x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V
 \end{aligned}$$

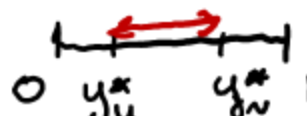
-
- 1: Pick τ in $(0, 1)$ uniformly at random.
 - 2: **for all** $v \in V$ **do**
 - 3: Set $y_v = 1$ if $y_v^* > \tau$, 0 otherwise
 - 4: **return** y .
-

comes from optimal solution
to LP relaxation

Why? Fix $e = (u, v) \in E$

e is cut iff $y_u = 0$ and $y_v = 1$

$\Leftrightarrow y_u^* \leq \tau < y_v^*$



$$\begin{aligned}
 &\Pr[y_u^* \leq \tau < y_v^*] \\
 &= y_v^* - y_u^* \\
 &\leq x_{uv}^*
 \end{aligned}$$

LP Rounding: st-Min-CUT

$$\begin{aligned}
 \mathbb{E}[\text{value}(S)] &= \sum_{e \in E} c(e) \mathbb{E}[\mathbb{1}_{e \text{ is cut}}] \\
 &\stackrel{\substack{\uparrow \\ \text{defined} \\ \text{by } y}}{=} \sum_{e \in E} c(e) \Pr[e \text{ is cut}] \\
 &\leq \sum_{e \in E} c(e) x_e^* \quad (\text{Previous slide}) \\
 &= -\text{OPT}_{\text{LP}} \\
 &\leq -\text{OPT}_{\text{ILP}} \\
 &= \text{value of min-cut} \quad \square
 \end{aligned}$$

Conclusion: $\mathbb{E}[\text{value}(S)] = \text{value of min-cut}$

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V$$

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V$$

LP and ILP in practice

- <https://au.mathworks.com/help/optim/ug/linprog.html>
- <https://au.mathworks.com/help/optim/ug/intlinprog.html>
- <https://reference.wolfram.com/language/ref/LinearProgramming.html>
- <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>
- <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html>
- [...]

ILP+LP Relaxation+Rounding: Max-SAT

Variables $x_1, \dots, x_n \in \{\underbrace{\text{true}}_{=1}, \underbrace{\text{false}}_{=0}\}$

$\phi = C_1 \wedge C_2 \wedge \dots \wedge C_m$ where

clause $C_j = x_{i_1} \vee \bar{x}_{i_2} \vee \dots \vee x_{i_{\ell_j}}$ $\ell_j = |C_j|$

* SAT: is there a way to set $x_1, \dots, x_n$ s.t. $\phi \equiv \text{true}$?
(NP-complete)

* MAX-SAT: how many C_j 's can we make (simultaneously) true?
(NP-hard)

Want to approximate this number.

[Don't know how to approx this better than 0.878-approx]

~~ILP+LP Relaxation+Rounding: Max-SAT~~

Theorem 47. *The “obvious” randomised algorithm which sets each variable x_i independently and uniformly at random gives, in expectation, a $\frac{1}{2}$ -approximation for MAX-SAT.*

~~ILP+LP Relaxation+Rounding: Max-SAT~~

Theorem 47. The "obvious" randomised algorithm which sets each variable x_i independently and uniformly at random gives, in expectation, a $\frac{1}{2}$ -approximation for MAX-SAT.

Proof Fix $1 \leq j \leq m$, Consider C_j .

$$C_j = x_{i_1} \vee \bar{x}_{i_2} \vee \bar{x}_{i_3} \vee \dots \vee x_{i_{37}}$$

$$l_j = |C_j| = 37$$

↑
here

$$\Pr[C_j \text{ is not satisfied}] = \frac{1}{2^{l_j}} \quad (\text{all literals must go the wrong way})$$

$$\mathbb{E}[\text{value}] = \sum_{j=1}^m \Pr[C_j \text{ is satisfied}] = \sum_{j=1}^m \left(1 - \frac{1}{2^{l_j}}\right) \geq \sum_{j=1}^m \left(1 - \frac{1}{2}\right) = \frac{m}{2}$$

(l_j ≥ 1) □

↑
want to
analyse
 $\sum_{j=1}^m \Pr[C_j \text{ is satisfied}]$

~~ILP+LP Relaxation+Rounding: Max-SAT~~

Theorem 47. The “obvious” randomised algorithm which sets each variable x_i independently and uniformly at random gives, in expectation, a $\frac{1}{2}$ -approximation for MAX-SAT.

Pathetic, but would be better if $l_j \gg 1$ for all j
(even $l_j \geq 3$ would give $\frac{7}{8}$ -approx)

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

is x_i true? $\rightarrow$ $y_i \in \{0, 1\}$
 $z_j \in \{0, 1\}$
 $\uparrow$ is C_j satisfied?

$$\forall 1 \leq i \leq n$$

$$\forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$y_i \in \{0, 1\}$$

$$\forall 1 \leq i \leq n$$

$$z_j \in \{0, 1\}$$

$$\forall 1 \leq j \leq m$$

Claim: The ILP is equivalent to MAX-SAT.

⌚. $\Rightarrow$ Given assignment x to MAXSAT,
get feasible solution y, z to ILP
with same value. (set $y_i = 1$ iff $x_i = \text{true}$)

$\Leftarrow$ Given optimal solution (y, z) to ILP,
get assignment x to MAXSAT
with same value. (set $x_i = \text{true}$ iff $y_i = 1$)

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$y_i \in \{0, 1\}$$

$$\forall 1 \leq i \leq n$$

$$z_j \in \{0, 1\}$$

$$\forall 1 \leq j \leq m$$

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1$$

$$\forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1$$

$$\forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

How to round? Given optimal solution y^*, z^* to LP,
 y_i^* is not in $\{0, 1\}$
 but in $[0, 1]$



Set $x_i = \begin{cases} 0 \\ 1 \end{cases}$ w.p $1 - y_i^*$
 w.p y_i^*
 independently.

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$y_i \in \{0, 1\} \quad \forall 1 \leq i \leq n$$

$$z_j \in \{0, 1\} \quad \forall 1 \leq j \leq m$$

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1 \quad \forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1 \quad \forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

Input: Instance $\phi = (C_1, \dots, C_m)$ of MAX-SAT on n variables

- 1: Solve the LP relaxation (Fig. 16), getting solution (y^*, z^*) .
 - 2: **for all** $1 \leq i \leq n$ **do**
 - 3: Set $x_i = 1$ with probability y_i^* , independently of others.
 - 4: **return** x .
-

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$y_i \in \{0, 1\} \quad \forall 1 \leq i \leq n$$

$$z_j \in \{0, 1\} \quad \forall 1 \leq j \leq m$$

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1 \quad \forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1 \quad \forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

Theorem 48. *The randomised rounding given in Algorithm 20 gives, in expectation, a $(1 - \frac{1}{e})$ -approximation for MAX-SAT.*

↑
what is this
doing here?!

$$1 - \frac{1}{e} \approx 0.632$$

Detour: the AM-GM inequality

Fact (AM-GM). For any $a_1, a_2, \dots, a_k \geq 0$,

$$\sqrt[k]{a_1 a_2 \dots a_k} \leq \frac{a_1 + a_2 + \dots + a_k}{k}$$

$$\begin{aligned} k=2 \quad & (b_1 - b_2)^2 \geq 0 \\ \Leftrightarrow \quad & \frac{b_1^2 + b_2^2}{2} \geq b_1 b_2 \end{aligned}$$

ILP+LP Relaxation+Rounding: Max-SAT

$$\mathbb{E}[\text{value}_{\phi}(x)] = \sum_{j=1}^m \Pr[C_j \text{ is satisfied}]$$

$$= \sum_{j=1}^m (1 - \Pr[C_j \text{ not satisfied}])$$

Fix x
 S : set of
 variables
 not negated
 in C_j

$$\Pr[C_j \text{ not satisfied}] = \prod_{i \in S} (1 - y_i^*) \cdot \prod_{i \notin S} y_i^*$$

$$\stackrel{\text{AM-GM}}{\leq} \left(\frac{1}{\ell_j} \left(\sum_{i \in S} (1 - y_i^*) + \sum_{i \notin S} y_i^* \right) \right)^{\ell_j}$$

$$= \left(\frac{1}{\ell_j} \left(\ell_j - \underbrace{\left(\sum_{i \in S} y_i^* + \sum_{i \notin S} (1 - y_i^*) \right)}_{\geq z_j} \right) \right)^{\ell_j}$$

maximise $\sum_{j=1}^m z_j$

subject to

$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$

$0 \leq y_i \leq 1$

$\forall 1 \leq i \leq n$

$0 \leq z_j \leq 1$

$\forall 1 \leq j \leq m$

ILP+LP Relaxation+Rounding: Max-SAT

$$\forall j \quad \Pr[C_j \text{ not satisfied}] \leq \left(1 - \frac{z_j^*}{\ell_j}\right)^{\ell_j}$$

$$\underline{\text{so}} \quad \mathbb{E}[\text{value}_\phi(x)] \geq \sum_{j=1}^m \left(1 - \left(1 - \frac{z_j^*}{\ell_j}\right)^{\ell_j}\right) \quad * ?!$$

$$\stackrel{(+)}{\geq} \sum_{j=1}^m \left(1 - \left(1 - \frac{1}{\ell_j}\right)^{\ell_j}\right) z_j^*$$

$$\geq \inf_{\ell \geq 1} \left(1 - \left(1 - \frac{1}{\ell}\right)^\ell\right) \sum_{j=1}^m z_j^*$$

$$= \left(1 - \frac{1}{e}\right) \text{OPT}_{LP} \quad \text{"of course!"}$$

$$\stackrel{\text{FACT}}{\geq} \left(1 - \frac{1}{e}\right) \text{OPT}_{ILP} = \left(1 - \frac{1}{e}\right) \text{OPT}_\phi \quad \square$$

$$\text{maximise } \sum_{j=1}^m z_j \quad *$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

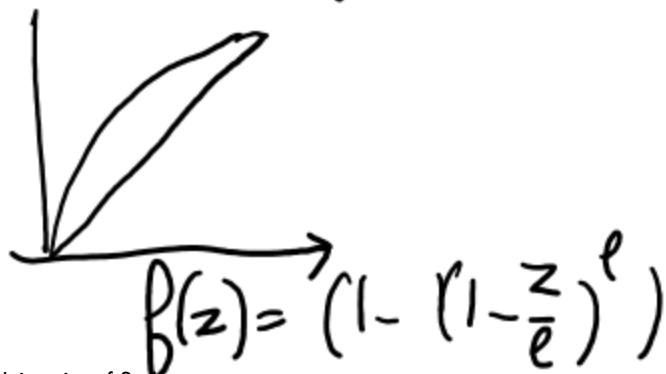
$$0 \leq y_i \leq 1$$

$$\forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1$$

$$\forall 1 \leq j \leq m$$

(+) By convexity
(here, concavity)



ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1$$

$$\forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1$$

$$\forall 1 \leq j \leq m$$

Good for small clauses!

Max-SAT: Can we do better?

Yes
Take the best of the two
 $(\frac{1}{2})$ -approx and $(1 - \frac{1}{e})$ -approx

Max-SAT: Can we do better?

Theorem. The “best-of-two” approach which runs both the naïve randomised algorithm and the randomised rounding gives, in expectation, a $\frac{3}{4}$ -approximation for Max-SAT.

$$\frac{3}{4} > 1 - \frac{1}{e}$$

Max-SAT: Can we do better?

Theorem. The “best-of-two” approach which runs both the naïve randomised algorithm and the randomised rounding gives, in expectation, a $3/4$ -approximation for Max-SAT.

$$\begin{aligned}
 \mathbb{E}[\max(\text{val}_\phi(x^{(1)}), \text{val}_\phi(x^{(2)}))] &\stackrel{(*)}{\geq} \frac{1}{2} \mathbb{E}[\text{val}_\phi(x^{(1)}) + \text{val}_\phi(x^{(2)})] \\
 &\stackrel{(**)}{\geq} \frac{1}{2} \sum_{j=1}^m \left(\left(1 - \frac{1}{2^{l_j}}\right) + \left(1 - \left(1 - \frac{1}{2^{l_j}}\right)^{l_j}\right) z_j^* \right) \\
 &\stackrel{(***)}{\geq} \frac{1}{2} \sum_{j=1}^m \underbrace{\left(\left(1 - \frac{1}{2^{l_j}}\right) + \left(1 - \left(1 - \frac{1}{2^{l_j}}\right)^{l_j}\right) z_j^* \right)}_{\triangleq g(l_j)} z_j^* \\
 &\stackrel{(***)}{\geq} \frac{3}{4} \sum_{j=1}^m z_j^* \stackrel{\text{OPT}_{LP}}{\geq} \frac{3}{4} \text{OPT}_{LP}
 \end{aligned}$$

$(*)$ max $\geq$ average
 $(**)$ previous analysis
 $(***)$ “magic”

Claim: $g(l) \geq \frac{3}{4}$
 for all $l \in \{1\} \cup [2, \infty)$

can add z_j^* here, since $z_j^* \leq 1$

Recap

- LPs are **powerful** (but **not enough**)
- We know how to solve them **efficiently** (in theory and practice)
- ILPs are **more powerful**
- We **don't know how to solve them** (though we often can in practice)

